

IPD434

Redes neuronales

De evaluación de datos a arquitecturas profundas

Dr. Patricio Olivares Roncagliolo

Conexión con el curso

- Los sistemas difusos codifican conocimiento mediante membresías y reglas.
- Los algoritmos evolutivos buscan soluciones mediante poblaciones.
- Las redes neuronales ajustan parámetros a partir de ejemplos.

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{n} \xrightarrow{\text{entrenamiento}} f_\theta(x)$$

> El modelo aprende regularidades de los datos. No aprende automáticamente el objetivo real ni corrige un diseño experimental deficiente.

Objetivos de la unidad

Al finalizar se espera poder:

1. **Formular** tareas de clasificación y regresión.
2. **Separar** entrenamiento, validación y prueba sin fuga de información.
3. **Seleccionar** métricas coherentes con el costo de los errores.
4. **Explicar** perceptrón, descenso de gradiente y retropropagación.
5. **Diseñar** una red multicapa con regularización.
6. **Distinguir** cuándo utilizar capas densas, recurrentes o convolucionales.

Ruta de la clase

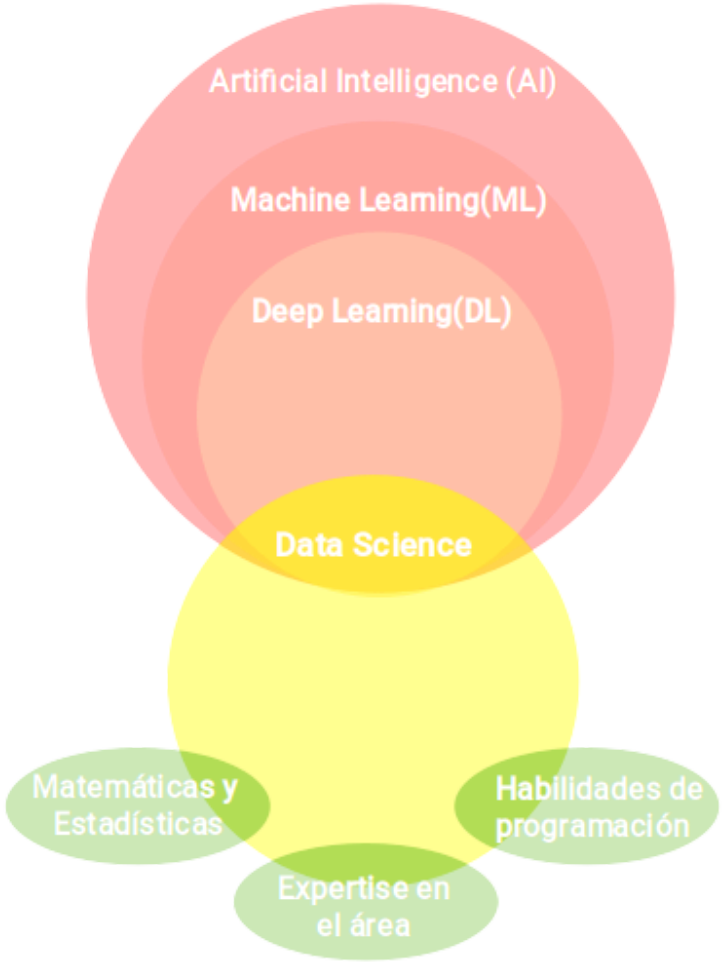
1. Definimos la tarea, los datos y el costo de equivocarse.
2. Separamos los datos y elegimos métricas antes de entrenar.
3. Construimos desde el perceptrón hasta una red multicapa.
4. Explicamos cómo descenso de gradiente y retropropagación ajustan sus parámetros.
5. Controlamos el sobreajuste y elegimos una arquitectura acorde con la estructura de los datos.
6. Cerramos con un protocolo de evaluación reproducible.

Aprendizaje Automático

El aprendizaje automático (*Machine Learning*) reúne métodos que ajustan un modelo a partir de ejemplos para producir predicciones o decisiones sobre observaciones nuevas.

> Aprender significa capturar regularidades de los datos para generalizar a observaciones que todavía no hemos visto.

Aprendizaje Automático



¿Cuándo usar aprendizaje automático?

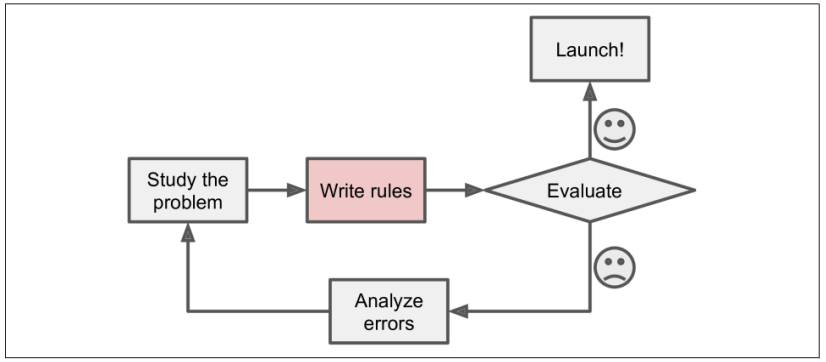


Figure 1-1. The traditional approach

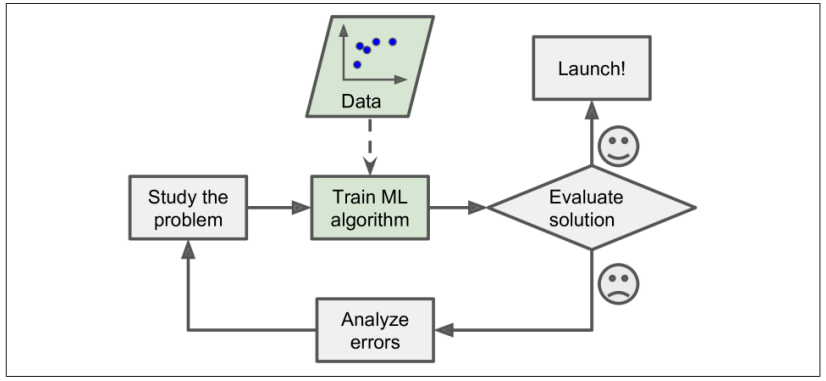


Figure 1-2. Machine Learning approach

Programación explícita

Se debe anticipar los patrones, escribir las reglas y mantener sus excepciones.

Aprendizaje desde ejemplos

Un modelo ajusta sus parámetros para capturar regularidades presentes en los datos.

Qué solución usar depende de si existe una regla simple que permita resolver el problema, o si los datos disponibles permiten obtener una relación generalizable.

Bibliotecas de Aprendizaje Automático



- **NumPy y Pandas:** representar y preparar datos.
- **Scikit Learn:** modelos clásicos, particiones y métricas.
- **PyTorch:** construir y entrenar redes neuronales.

> La biblioteca facilita el trabajo, pero la tarea, los datos y la métrica siguen siendo decisiones de diseño.

Scikit Learn

Scikit Learn es una biblioteca de Python para aprendizaje automático. Incluye modelos de clasificación y regresión, herramientas para preparar datos y métricas para evaluar resultados.

Instalación

```
pip install -U scikit-learn
```

Enlaces

Sitio oficial

Guía de instalación

> Con la biblioteca instalada, el trabajo se organiza en objetos con responsabilidades distintas.

Scikit Learn

Una ventaja de Scikit Learn es la consistencia de su API: sus objetos siguen convenciones comunes y separan sus responsabilidades.

Objeto	Responsabilidad	Método típico
Estimador	Ajustar parámetros	`fit(X, y)`
Transformador	Cambiar la representación	`fit` y `transform`
Predictor	Producir salidas	`predict(X)`

Un flujo típico encadena: **ajustar con entrenamiento → transformar sin volver a ajustar → predecir sobre datos nuevos**.

Así, podemos cambiar de modelo sin rehacer toda la lógica experimental: `fit` ajusta, `transform` transforma y `predict` predice.

Tipos de Aprendizaje Automático

Los paradigmas se distinguen por la información que guía el aprendizaje.

Tipo	Información disponible	Qué busca el modelo	Ejemplo
Supervisado	Entradas con objetivo conocido	Predecir el objetivo de un caso nuevo	Estimar consumo eléctrico desde mediciones históricas
No supervisado	Datos sin objetivo	Encontrar grupos o estructura	Agrupar usuarios según su patrón de uso
Semisupervisado	Pocas etiquetas y muchos datos sin etiqueta	Aprovechar ambos conjuntos	Clasificar imágenes médicas con pocas etiquetas
Por refuerzo	Estados, acciones y recompensas	Elegir acciones con mayor retorno	Controlar el uso de energía de un sistema

Aprendizaje Supervisado

Cada ejemplo de entrenamiento contiene una entrada y su objetivo conocido:

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{n}$$

El modelo ajusta sus parámetros para que, dada una entrada nueva x, su predicción $f_\theta(x)$ se aproxime al objetivo esperado y.

> La forma del objetivo y define la tarea. Puede representar una categoría o una cantidad.

Tipos de Aprendizaje Supervisado

El objetivo y determina qué debe predecir el modelo.

Tarea	Forma del objetivo	Pregunta	Ejemplos
Clasificación	Categoría o etiqueta	¿A qué clase pertenece?	Detectar una falla, filtrar spam, identificar una prenda
Regresión	Cantidad numérica	¿Qué valor se espera?	Estimar consumo eléctrico, precio o temperatura

Esta diferencia define la salida del modelo, la función de pérdida y la métrica de evaluación.

Clasificación: predecir una categoría

Cada objetivo pertenece a una de K clases posibles:

$$f_\theta : \mathbb{R}^d \to \{1, \ldots, K\}$$

- **Clasificación binaria:** dos clases posibles, por ejemplo `falla` y `sin falla`.
- **Clasificación con varias clases:** una clase entre varias posibles, como `manzana`, `plátano` o `naranja`.

> Un filtro de correo puede clasificar un mensaje como "spam" o "no spam". El objetivo es asignar una etiqueta, no medir una cantidad.

Regresión: predecir una cantidad

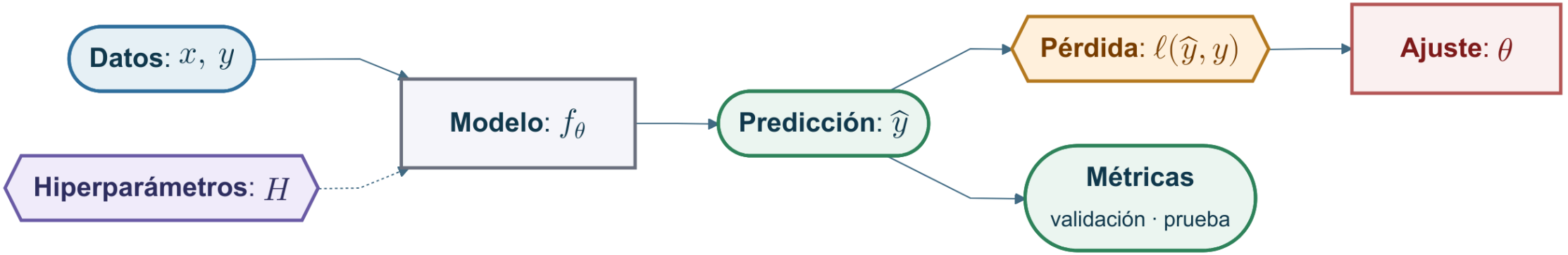
El objetivo es un valor numérico continuo, o un vector de valores:

$$f_\theta : \mathbb{R}^d \to \mathbb{R}^m$$

Por ejemplo, un modelo puede estimar el consumo eléctrico de la próxima hora en kWh, la temperatura de un equipo en $^\circ\mathrm{C}$ o el precio de una vivienda en millones.

> Un pronóstico de consumo de 120 kWh no pertenece a una clase. Debe compararse con el consumo real y el error conserva la unidad del problema.

Componentes de un experimento supervisado



> La pérdida guía el ajuste de θ con los datos de entrenamiento. La validación permite elegir hiperparámetros y la prueba se reserva para reportar el desempeño final.

Protocolo de entrenamiento y evaluación supervisada

La salida $\hat{y}$ define si la tarea es clasificación o regresión. El protocolo experimental define cómo se ajusta el modelo y cómo se estima su desempeño fuera de los datos usados para decidir.

Componente	Rol formal	Se desarrolla en
Parámetros θ y pérdida ℓ	Ajustar el modelo con entrenamiento	Pérdida promedio
Datos y preprocesamiento	Preservar la información disponible sin fuga	Particiones y preprocesamiento
Hiperparámetros $\mathcal{H}$	Elegir una configuración con validación	Selección de modelos
Métricas	Cuantificar la calidad de las predicciones	Evaluación de clasificación y regresión

Cómo se ajustan los parámetros del modelo

Sea $\mathcal{D}_{\text{train}} = \{(x_i, y_i)\}_{i=1}^{n_{\text{train}}}$. Cada ejemplo produce un error mediante la pérdida $\ell(\hat{y}, y)$

.

$$\widehat{L}_{\text{train}}(\theta) = \frac{1}{n_{\text{train}}} \sum_{i=1}^{n_{\text{train}}} \ell\big(f_\theta(x_i), y_i\big)$$

El entrenamiento resume los errores individuales en una pérdida promedio y busca los parámetros que la minimizan:

$$\hat{\theta} = \arg\min_{\theta} \widehat{L}_{\text{train}}(\theta)$$

Esta optimización responde cuánto se equivoca el modelo en los ejemplos disponibles para aprender. Un error bajo allí no garantiza un buen desempeño en ejemplos nuevos.

Parámetros e hiperparámetros en el aprendizaje

Los **parámetros** son valores internos que el modelo aprende, como pesos y sesgos.
Los **hiperparámetros** configuran el modelo y su entrenamiento, como arquitectura, tasa de aprendizaje, tamaño de lote y regularización.

$\mathcal{A}_{\mathbf{h}}$ es el procedimiento de entrenamiento definido por $\mathbf{h}$. Usa $\mathcal{D}_{\text{train}}$ para calcular gradientes y actualizar los parámetros.

$$\hat{\theta}(\mathbf{h}) = \mathcal{A}_{\mathbf{h}}\left(\mathcal{D}_{\text{train}}\right)$$

$\hat{\theta}(\mathbf{h})$ son los parámetros finales aprendidos con esa configuración.

Selección de hiperparámetros mediante validación

Cada configuración $\mathbf{h}$ produce una función entrenada $f_{\hat{\theta}(\mathbf{h})}$, definida por los parámetros obtenidos. La validación permite comparar esas funciones sin usar todavía el conjunto de prueba:

$$\mathbf{h}^* = \arg\min_{\mathbf{h}\in\mathcal{H}} \widehat{L}_{\text{val}}\left(\hat{\theta}(\mathbf{h})\right)$$

- $\mathcal{H}$: conjunto de configuraciones candidatas.
- $\widehat{L}_{\text{val}}$: pérdida promedio calculada sobre validación.
- $\mathbf{h}^*$: configuración cuyo modelo obtiene la menor pérdida de validación.

La prueba se reserva para estimar el desempeño final de la configuración seleccionada.

Particiones para ajuste, selección y evaluación

El conjunto de datos se divide en subconjuntos disjuntos:

$$\mathcal{D} = \mathcal{D}_{\text{train}} \dot{\cup} \mathcal{D}_{\text{val}} \dot{\cup} \mathcal{D}_{\text{test}}$$

Partición	Uso	¿Ajusta decisiones?
Entrenamiento	Estimar parámetros	Sí
Validación	Elegir hiperparámetros y detener	Sí
Prueba	Estimar desempeño final	No

> Preprocesar con estadísticas calculadas sobre todos los datos, seleccionar modelos mirando prueba o duplicar muestras entre particiones produce fuga de información.

Porcentajes habituales de partición

Los porcentajes son orientativos. Elija la proporción según el tamaño de los datos y la precisión necesaria.

Estructura	Proporción	Cuándo usarla
Entrenamiento / prueba	80 / 20	Caso general o validación cruzada
Entrenamiento / prueba	70 / 30 o 60 / 40	Dataset grande con prueba más estable
Entrenamiento / validación / prueba	80 / 10 / 10	Ajuste moderado de parámetros
Entrenamiento / validación / prueba	70 / 15 / 15	Dataset grande o muchas configuraciones
Entrenamiento / validación / prueba	90 / 5 / 5	Dataset muy grande. El 5% debe bastar para validar y probar

Preprocesamiento

Un transformador, por ejemplo un escalador o codificador, debe estimarse **únicamente** en $\mathcal{D}_{\text{train}}$:

$$\widehat{T} = \text{fit}(X_{\text{train}}), \qquad X'_s = \widehat{T}(X_s), \quad s \in \{\text{train}, \text{val}, \text{test}\}$$

Esto permite aplicar la misma representación a cada partición sin incorporar información de validación o prueba durante el ajuste.

Secuencia de selección y prueba final

1. **Dividir** los datos respetando grupos, tiempo y clases cuando corresponda.
2. **Ajustar** el preprocesamiento solo con entrenamiento.
3. **Transformar** entrenamiento, validación y prueba con esos mismos parámetros.
4. **Seleccionar** arquitectura e hiperparámetros usando validación.
5. **Evaluar** la prueba una sola vez al final.

Métricas de evaluación

Una métrica resume la calidad de las predicciones comparando los objetivos reales y con las predicciones $\hat{y}$:

$$M\left(\mathbf{y}, \hat{\mathbf{y}}\right) \in \mathbb{R}$$

La métrica debe responder a la decisión que se quiere apoyar. Por eso importa:

- qué error se considera relevante
- si las clases están desbalanceadas
- qué unidades tiene el objetivo
- qué costo tiene equivocarse

Una misma predicción puede parecer buena o mala según la métrica utilizada.

Métricas para clasificación

En clasificación, la predicción es una etiqueta o un puntaje que luego se convierte en etiqueta mediante un umbral.

En clasificación binaria, la calidad del modelo se describe contando aciertos y errores: verdaderos positivos (TP), verdaderos negativos (TN), falsos positivos (FP) y falsos negativos (FN).

Clasificación binaria: aciertos y errores

Supongamos que la clase positiva representa la condición que queremos detectar:

Resultado	Condición real	Predicción	Interpretación	Ejemplo
TP	Positiva	Positiva	Acierto al detectar la condición	 esto es 5
TN	Negativa	Negativa	Acierto al descartar la condición	 esto no es 5
FP	Negativa	Positiva	Falsa alarma	 esto no es 5
FN	Positiva	Negativa	Omisión de un caso real	 esto es 5

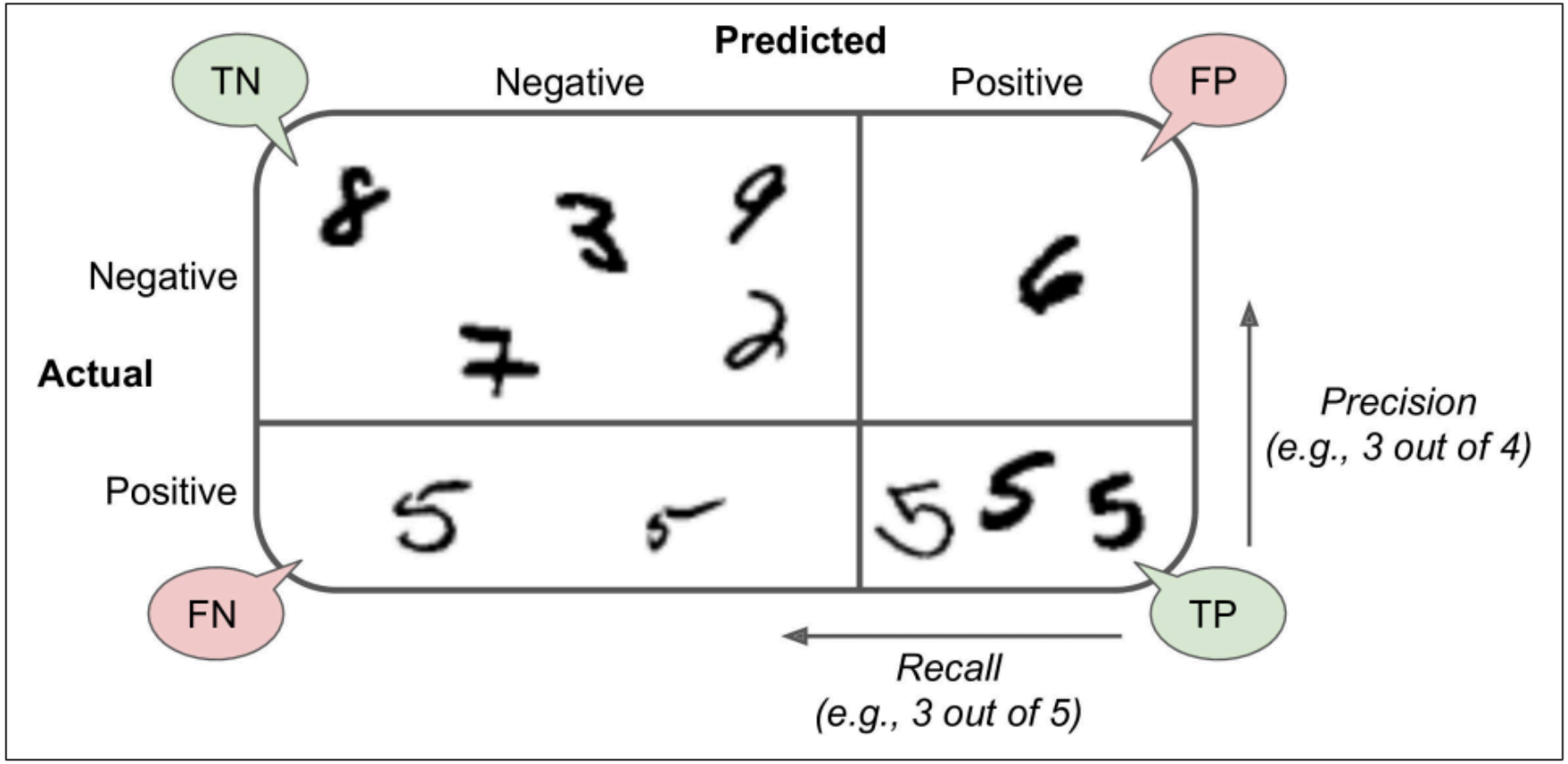
El significado de "positivo" depende de la decisión. No tiene por qué corresponder a la clase más frecuente.

Matriz de confusión binaria

Una matriz de confusión cruza la clase real con la clase predicha y cuenta cuántos casos caen en cada combinación.

En clasificación binaria, sus cuatro celdas corresponden a TP, TN, FP y FN. Así permite observar por separado los aciertos y los tipos de error.

Matriz de confusión binaria



Cómo leer una matriz de confusión

La matriz cruza dos perspectivas sobre los mismos casos:

- **Filas:** condición real observada.
- **Columnas:** clase predicha por el modelo.
- **Diagonal:** aciertos TP y TN.
- **Fuera de la diagonal:** errores FP y FN.

La matriz conserva los conteos necesarios para calcular exactitud, precisión, exhaustividad y especificidad. La importancia de cada celda depende del problema.

Cálculo de métricas a partir de una matriz de confusión

El notebook 04_clasificacion_titanic.ipynb reproduce el cálculo sobre el dataset Titanic: Machine Learning from Disaster. Sobre sus 179 casos de prueba:

	Predice positivo	**Predice negativo**
Real positivo	$TP = 59$	$FN = 17$
Real negativo	$FP = 16$	$TN = 87$

- Exactitud: $(59 + 87)/179 = 81{,}6\%$.
- Precisión: $59/(59 + 16) = 78{,}7\%$.
- Exhaustividad: $59/(59 + 17) = 77{,}6\%$.

La misma matriz puede verse "bien" o "mal" según qué error sea más costoso.

Exactitud

Exactitud (*accuracy*)

Aciertos sobre el total de casos:

$$\text{Acc} = \frac{TP + TN}{TP + TN + FP + FN}$$

- ¿Qué proporción del total fue clasificada correctamente?
- Usar cuando las clases y los costos de error son similares.

Especificidad

Especificidad (*specificity*, TNR)

Negativos reales reconocidos correctamente:

$$\text{Esp} = \frac{TN}{TN + FP}$$

- ¿Qué proporción de los negativos reales fue descartada correctamente?
- Usar cuando importa no marcar como positivos los negativos reales y una falsa alarma FP resulta costosa.

Precisión

Precisión (*precision*, PPV)

Positivos reales entre las predicciones positivas:

$$\text{Prec} = \frac{TP}{TP + FP}$$

- ¿Cuántos casos marcados como positivos eran realmente positivos?
- Usar cuando una falsa alarma FP es costosa en relación con omitir positivos.

Exhaustividad

Exhaustividad (*recall*, sensibilidad, TPR)

Positivos reales detectados por el modelo:

$$\text{Rec} = \frac{TP}{TP + FN}$$

- ¿Cuántos positivos reales fueron detectados?
- Usar cuando una omisión FN es costosa en relación con generar falsas alarmas.

Limitaciones de las métricas

Consideremos 20 números: 3 son 5 y 17 no son 5.

El modelo "nada es 5" predice "no es 5" para todos los casos:

$$TP = 0, \qquad TN = 17, \qquad FP = 0, \qquad FN = 3$$

Su exactitud es:

$$\text{Acc} = \frac{0 + 17}{20} = 85\%$$

El valor parece alto, pero el modelo no detecta ninguno de los tres cincos.

Limitaciones de las métricas

Para el modelo "nada es 5", las otras métricas muestran un comportamiento muy diferente:

$$\text{Rec} = \frac{0}{0+3} = 0, \qquad \text{Esp} = \frac{17}{17+0} = 1$$

La precisión es indefinida porque el modelo no predice ningún positivo. Una biblioteca puede aplicar una convención configurable para evitar esta división por cero:

$$\text{Prec} = \frac{0}{0+0} = \frac{0}{0}$$

La exactitud está dominada por la clase mayoritaria. Por eso debemos examinar qué tipo de acierto o error importa en el problema.

F_1: equilibrio entre precisión y exhaustividad

$$F_1 = 2\frac{\mathrm{Prec} \cdot \mathrm{Rec}}{\mathrm{Prec} + \mathrm{Rec}}$$

El F_1 es la media armónica de precisión y exhaustividad.

- Solo es alto cuando ambas métricas son altas.
- Es útil cuando precisión y exhaustividad tienen una importancia comparable.

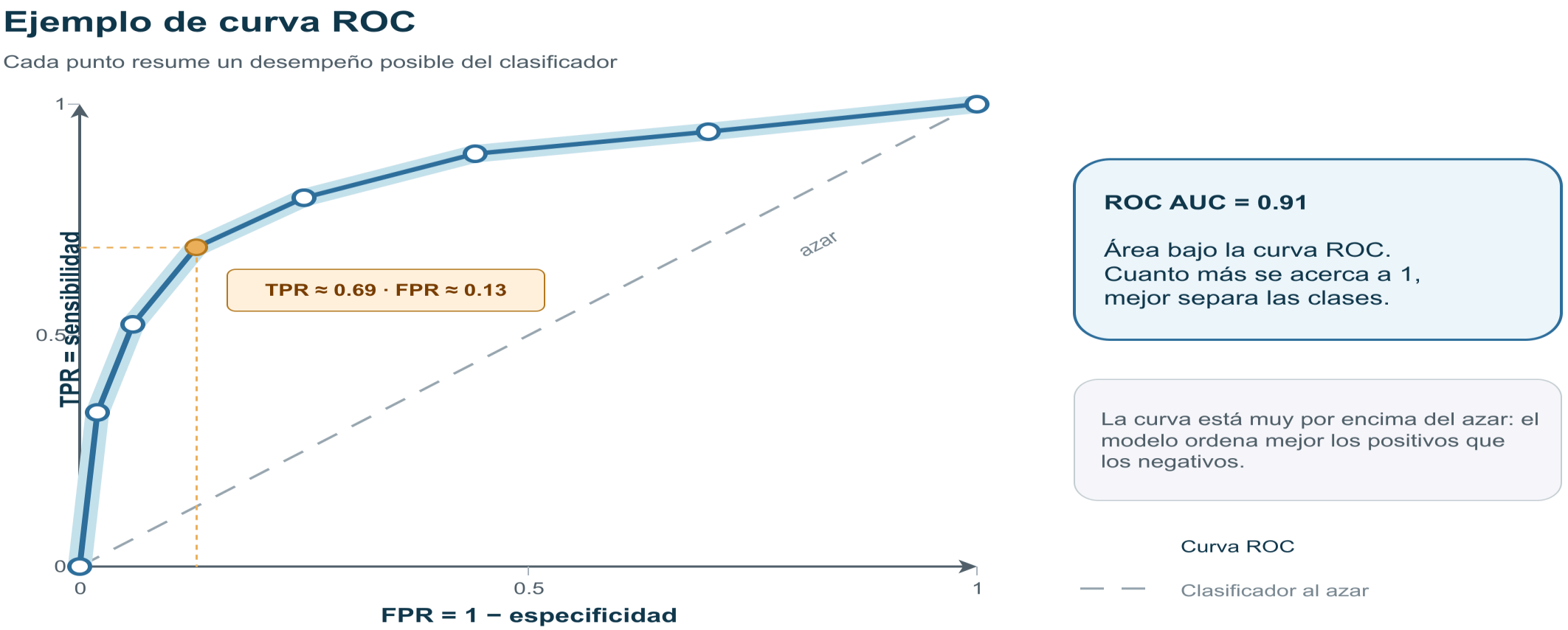
Curva ROC: interpretación de sus ejes

La curva ROC muestra cómo cambia el desempeño de un clasificador al modificar el umbral aplicado a su puntaje:

$$\mathrm{TPR} = \frac{TP}{TP + FN}, \qquad \mathrm{FPR} = \frac{FP}{FP + TN}$$

- **Eje vertical:** TPR o sensibilidad, proporción de positivos reales detectados.
- **Eje horizontal:** FPR, proporción de negativos reales clasificados erróneamente como positivos.
- **Cada punto:** una regla de decisión distinta, con un compromiso entre detección y falsas alarmas.
- **Interpretación:** el punto ideal es $(0, 1)$. La diagonal representa un clasificador equivalente al azar.

Curva ROC y ROC AUC



El ejemplo muestra un clasificador cuya curva se encuentra por encima del azar.

Visualización interactiva de una curva ROC

Métricas para regresión

En regresión, la predicción es una **cantidad numérica**. Las métricas comparan y y $\hat{y}$ mediante una noción de distancia o variación explicada.

MAE: Mean Absolute Error

$$MAE = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

El MAE es el error absoluto promedio. Cada unidad de error contribuye de la misma forma, sin importar si la predicción quedó por encima o por debajo del valor real.

- Usarlo cuando se necesita una medida directa e interpretable en las unidades del objetivo.
- Es menos sensible a valores extremos que las métricas cuadráticas.

MSE: Mean Square Error

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

El MSE eleva cada error al cuadrado. Por eso un error grande pesa mucho más que varios errores pequeños.

- Usarlo cuando los errores grandes son especialmente costosos o deben penalizarse con fuerza.
- Su unidad queda elevada al cuadrado y es sensible a valores extremos.

RMSE: Root Mean Square Error

$$RMSE = \sqrt{MSE}$$

El RMSE conserva las unidades de la variable objetivo y resume la magnitud típica del error, dando más peso a las desviaciones grandes.

- Usarlo cuando se quiere relacionar el desempeño con una tolerancia operacional.
- Mantiene la sensibilidad del MSE frente a valores extremos.

R^2: comparación con una línea base

El coeficiente de determinación compara el error cuadrático del modelo con el de predecir siempre la media del conjunto evaluado:

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}$$

$R^2 = 1$ indica ajuste perfecto. $R^2 = 0$ corresponde al predictor constante $\bar{y}$ en ese conjunto evaluado. Un valor negativo indica que el modelo rinde peor que esa línea base.

- Usarlo para comparar cuánto mejora el modelo respecto de una referencia simple.
- Complementarlo con MAE o RMSE para conocer el error en las unidades originales.

Modelos predictivos supervisados

Un modelo predictivo aprende una función parametrizada a partir de datos etiquetados:

$$f_\theta : \mathcal{X} \longrightarrow \mathcal{Y}, \qquad \hat{y} = f_\theta(\mathbf{x})$$

Los parámetros θ se ajustan con entrenamiento y luego se evalúan sobre datos no vistos.

- **Clasificación:** $\mathcal{Y}$ contiene etiquetas discretas.
- **Regresión:** $\mathcal{Y} \subseteq \mathbb{R}^m$ contiene valores numéricos.

La tarea fija el tipo de salida, la función de pérdida y las métricas de evaluación.

Ejemplos de modelos predictivos en Scikit Learn

Clasificación

- Regresión logística: `LogisticRegression`
- KNN: `KNeighborsClassifier`
- Árboles y bosques: `DecisionTreeClassifier`, `RandomForestClassifier`
- Support Vector Machine Classifier: `SVC`

Regresión

- Modelos lineales: `LinearRegression`, `Ridge`, `Lasso`
- KNN: `KNeighborsRegressor`
- Árboles y ensambles: `RandomForestRegressor`, `GradientBoostingRegressor`
- Support Vector Machine Regressor: `SVR`

Modelo de regresión lineal

Consideremos m ejemplos de entrenamiento y N características por ejemplo. Para incorporar el intercepto, definimos $x_0^{(i)} = 1$:

$$h_\theta\left(x^{(i)}\right) = \sum_{j=0}^{N} \theta_j x_j^{(i)} = \theta_0 + \sum_{j=1}^{N} \theta_j x_j^{(i)}$$

- θ_0 es el intercepto y θ_j es el peso de la característica j.
- $i = 1, \ldots, m$ identifica el ejemplo y $j = 1, \ldots, N$ la característica.

El modelo es lineal en los parámetros, aunque las características pueden provenir de transformaciones no lineales.

Mínimos cuadrados ordinarios

El entrenamiento busca los valores de $\theta_0, \ldots, \theta_N$ que minimizan el error cuadrático promedio:

$$\mathrm{MSE}(\theta) = \frac{1}{m} \sum_{i=1}^{m} \left(h_\theta\left(x^{(i)}\right) - y^{(i)} \right)^2$$

$$\hat{\theta}_0, \ldots, \hat{\theta}_N = \underset{\theta_0, \ldots, \theta_N}{\operatorname{argmin}} \, \mathrm{MSE}(\theta)$$

Cada residuo $h_\theta\left(x^{(i)}\right) - y^{(i)}$ mide el error de un ejemplo. Elevarlo al cuadrado penaliza más las desviaciones grandes.

Condición de óptimo en mínimos cuadrados

La derivada parcial respecto de cada parámetro es:

$$\frac{\partial \operatorname{MSE}(\theta)}{\partial \theta_j} = \frac{2}{m} \sum_{i=1}^{m} \left(h_\theta\left(x^{(i)}\right) - y^{(i)} \right) x_j^{(i)}, \qquad j = 0, \ldots, N$$

En el mínimo, todas las derivadas se anulan. Por tanto, los parámetros satisfacen las ecuaciones normales escalares:

$$\sum_{i=1}^{m} \left(h_\theta\left(x^{(i)}\right) - y^{(i)} \right) x_j^{(i)} = 0, \qquad j = 0, \ldots, N$$

Estas ecuaciones forman un sistema de $N + 1$ ecuaciones para $\theta_0, \ldots, \theta_N$. Resolverlo entrega los parámetros que minimizan el MSE.

Descenso de gradiente en regresión lineal

También podemos ajustar cada parámetro mediante actualizaciones iterativas:

$$\theta_j \leftarrow \theta_j - \eta \frac{\partial \, \mathrm{MSE}(\theta)}{\partial \theta_j}, \qquad j = 0, \ldots, N$$

- $\eta > 0$ es la tasa de aprendizaje.
- El gradiente indica cómo cambia el MSE al modificar cada parámetro.

La misma regla se extenderá a redes neuronales, donde no existe en general una solución cerrada.

Redes Neuronales

Las redes neuronales son una familia de modelos paramétricos ampliamente utilizada para aprender relaciones complejas y generar nuevas salidas a partir de patrones aprendidos.

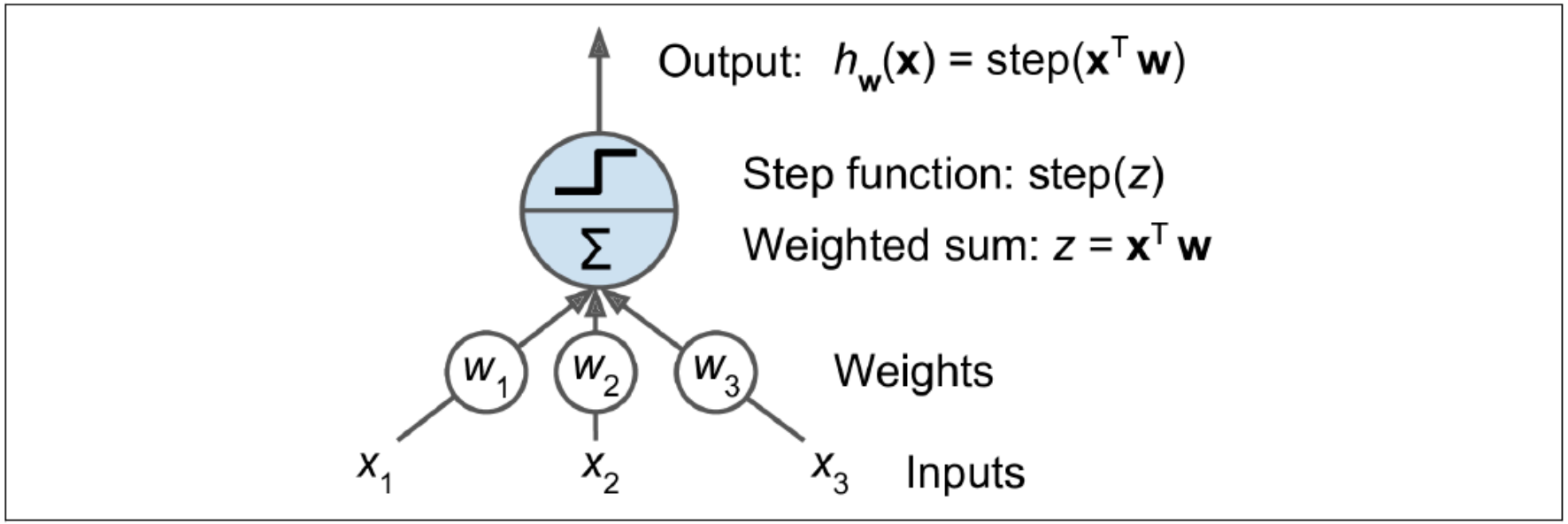
- **Predicción:** clasificación de imágenes, recomendación, pronóstico y detección de anomalías.
- **Modelos generativos:** creación de texto, imágenes, audio, video y código.
- **LLM:** modelos de lenguaje a gran escala que permiten asistentes como ChatGPT, Claude y otros sistemas conversacionales.

> La misma idea de apilar unidades simples permite construir modelos predictivos y generativos.

Ejemplos de aplicaciones de Redes Neuronales

- DeepMind: juegos y estrategia
- YOLOv3: detección de objetos
- Quick, Draw!: reconocimiento de dibujos
- Piedra, papel o tijera: reconocimiento de gestos
- ChatGPT: generación de texto
- Y una laaaaarga lista

Redes Neuronales: El Perceptrón



Redes Neuronales: El Perceptrón

$$z = w_0 + \sum_{j=1}^{N} w_j x_j, \qquad \hat{y} = \text{step}(z)$$

La función escalón convierte el puntaje en una etiqueta binaria:

$$\text{step}(z) = \begin{cases} 0, & z < 0 \\ 1, & z \geq 0 \end{cases}$$

Los pesos definen la orientación de la frontera y w_0 la desplaza. Una sola unidad solo puede separar clases linealmente separables.

Regresión lineal y perceptrón

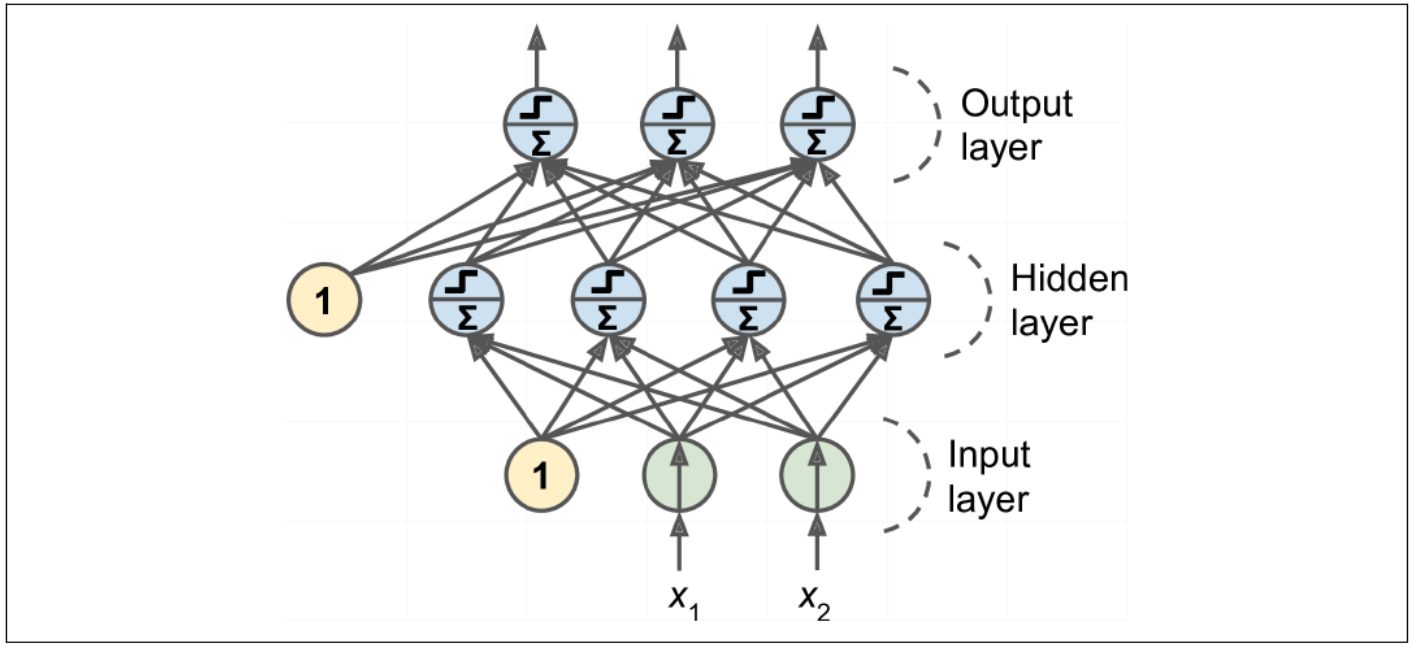
Ambos modelos calculan una combinación afín de las características. Cambian la interpretación de la salida:

Aspecto	Regresión lineal	Perceptrón
Puntaje	$h_\theta(x) = \theta_0 + \sum_{j=1}^{N} \theta_j x_j$	$z = w_0 + \sum_{j=1}^{N} w_j x_j$
Salida	$\hat{y} = h_\theta(x) \in \mathbb{R}$	$\hat{y} = \mathbb{I}\{z \geq 0\} \in \{0, 1\}$
Tarea	Predecir una cantidad	Separar dos clases
Ajuste habitual	MSE	Criterio del perceptrón o pérdida de clasificación
Geometría	Hiperplano de predicción	Hiperplano como frontera de decisión

Una sigmoide aplicada al puntaje produce una probabilidad diferenciable. Con entropía cruzada se obtiene regresión logística, una conexión directa con las redes neuronales.

Red neuronal multicapa

El perceptrón construye una sola frontera lineal. Una red multicapa encadena unidades para construir representaciones intermedias antes de producir la salida.



Red neuronal multicapa

- La **arquitectura** describe cuántas capas y unidades tiene la red, y cómo están conectadas.
- Una red con una o más capas ocultas es una red multicapa.
- Cuando hay muchas capas ocultas, se habla de una red profunda.

> Primero identificamos las capas y sus conexiones. Luego veremos por qué cada capa necesita una activación no lineal.

Tipos de capas

Capa	Rol	Características
Entrada	Recibe la representación de cada ejemplo	Tiene una unidad por característica. No transforma los datos por sí misma.
Oculta	Construye representaciones internas	Aplica pesos, sesgos y una activación. Puede haber una o varias.
Salida	Produce la predicción final	Su tamaño y activación dependen de la tarea.

Para una imagen, la entrada puede contener píxeles. Para datos tabulares, puede contener variables como edad, consumo o temperatura.

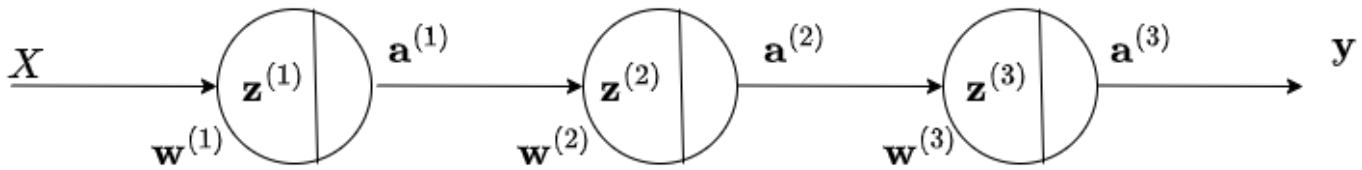
Capas densas: conexiones y cálculo

En una capa densa, cada unidad de una capa se conecta con todas las unidades de la capa siguiente. Cada conexión tiene un peso aprendido.

$$\mathbf{z}^{(\ell)} = \mathbf{W}^{(\ell)}\mathbf{a}^{(\ell-1)} + \mathbf{b}^{(\ell)}, \qquad \mathbf{a}^{(\ell)} = g^{(\ell)}\left(\mathbf{z}^{(\ell)}\right)$$

- $\mathbf{a}^{(0)} = \mathbf{x}$ es el vector de entrada.
- $\mathbf{W}^{(\ell)}$ contiene los pesos entre las capas $\ell - 1$ y ℓ.
- $\mathbf{b}^{(\ell)}$ contiene un sesgo por unidad de la capa ℓ.
- $g^{(\ell)}$ es la función de activación aplicada a cada componente.

Composición de capas



La salida de cada capa se convierte en la entrada de la siguiente. Así, la red encadena transformaciones hasta producir una predicción.

Profundidad y ancho

- **Profundidad:** número de capas ocultas. Aumentarla permite construir representaciones en varios niveles.
- **Ancho:** número de unidades en una capa. Afecta la cantidad de patrones que la capa puede representar.
- **Parámetros:** cada conexión y cada sesgo se ajustan durante el entrenamiento.

Más capas o más unidades aumentan la capacidad del modelo, pero también el costo de entrenamiento y el riesgo de sobreajuste. La arquitectura se selecciona con validación.

Función de activación

Una función de activación transforma el puntaje z de una unidad en su salida a:

$$z = \mathbf{w}^\top \mathbf{x} + b, a = g(z)$$

Su función principal es introducir no linealidad. Sin activaciones no lineales, varias capas densas seguidas se reducen a una sola transformación lineal.

> La activación de las capas ocultas permite representar relaciones que un perceptrón o una regresión lineal no pueden capturar por sí solos.

Activaciones lineal y ReLU

Lineal

$$g(z) = z$$

- Conserva el puntaje sin limitar su rango.
- Es habitual en la salida de una tarea de regresión.
- No agrega capacidad de representación en capas ocultas.

ReLU

$$g(z) = \max(0, z)$$

- Devuelve cero para puntajes negativos.
- Es una elección frecuente para capas ocultas.
- Una unidad puede quedar inactiva si recibe puntajes negativos de forma persistente.

Leaky ReLU

Leaky ReLU conserva una pendiente pequeña cuando el puntaje es negativo:

$$g(z) = \begin{cases} z, & z > 0 \\ \alpha z, & z \leq 0 \end{cases} \qquad \alpha > 0 \text{ pequeño}$$

- Es una variante de ReLU que reduce el riesgo de unidades inactivas.
- Para $z < 0$, la salida y el gradiente no son exactamente cero.
- Se usa en capas ocultas cuando ReLU presenta muchas unidades inactivas.

Sigmoide y tangente hiperbólica

Sigmoide

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

- Produce valores entre 0 y 1.
- En la salida binaria puede interpretarse como probabilidad.
- En capas ocultas puede producir gradientes muy pequeños para valores extremos.

Tangente hiperbólica

$$\tanh(z) = \frac{e^{z} - e^{-z}}{e^{z} + e^{-z}}$$

- Produce valores entre -1 y 1.
- Está centrada en cero.
- También puede saturarse para valores de gran magnitud.

Softmax para varias clases

Para K puntajes de salida $z_1, \ldots, z_K$, softmax produce una distribución de probabilidades:

$$\text{softmax}(\mathbf{z})_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}$$

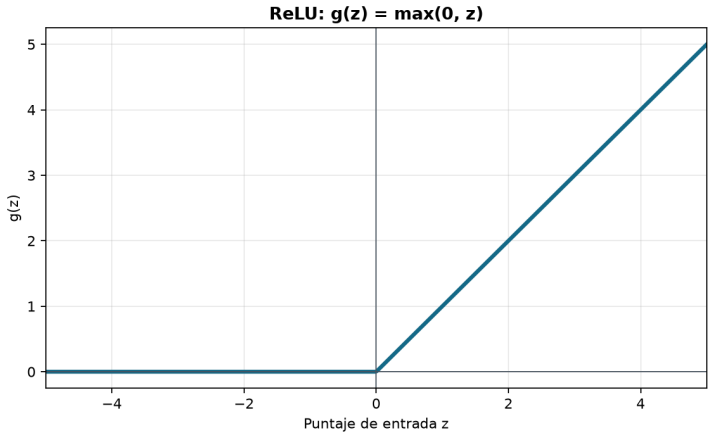
- Cada salida está entre 0 y 1.
- La suma de las K salidas es 1.
- La clase predicha suele ser la de mayor probabilidad.

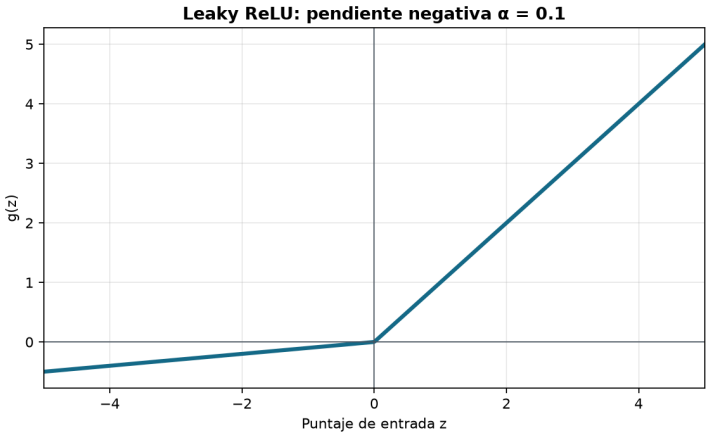
> Softmax se aplica en la capa de salida cuando hay varias clases mutuamente excluyentes.

Funciones de activación: forma y rango

Las curvas muestran cómo una activación transforma el puntaje z en una salida.

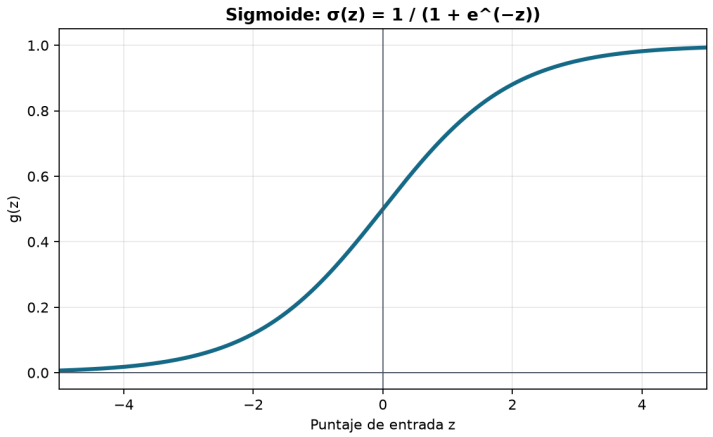


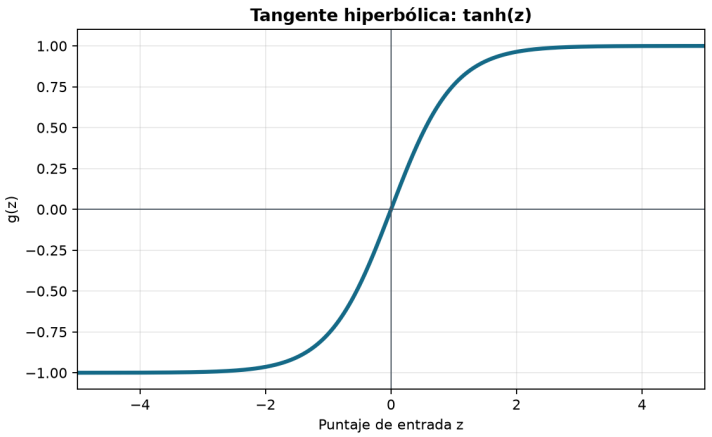


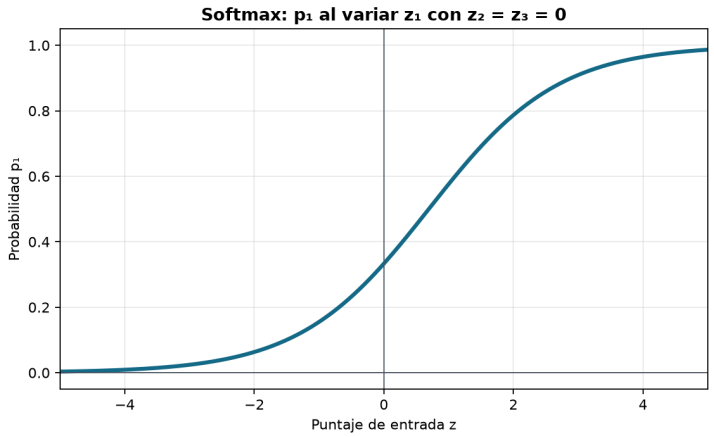


Sigmoide, tangente hiperbólica y softmax

Softmax depende de todos los puntajes. El gráfico muestra la probabilidad de una clase mientras varía su puntaje y los otros dos se mantienen fijos.







Selección de activación

Ubicación o tarea	Elección habitual	Motivo
Capas ocultas	ReLU o Leaky ReLU	Introducen no linealidad y suelen facilitar el entrenamiento.
Salida de regresión	Lineal	Permite predecir valores sin restringir su rango.
Salida binaria	Sigmoide	Produce un puntaje entre 0 y 1.
Salida con varias clases	Softmax	Distribuye la probabilidad entre las clases.

La activación se elige junto con la pérdida y la tarea. Por ejemplo, sigmoide y entropía cruzada binaria forman una combinación habitual.

Salida, etiquetas y función de pérdida

Tarea	Salida del modelo	Pérdida habitual
Regresión	Valor lineal $\hat{y} \in \mathbb{R}$	MSE o MAE
Clasificación binaria	*Logit* o probabilidad sigmoide	Entropía cruzada binaria
Clasificación con varias clases	Vector de *logits*	Entropía cruzada para varias clases

La codificación de etiquetas, la activación de salida y la pérdida deben ser compatibles. En PyTorch, `CrossEntropyLoss` recibe *logits* y aplica internamente la normalización necesaria.

Entropía cruzada y *softmax* en clasificación con varias clases

Para una etiqueta con una sola clase activa $\mathbf{y}$ y probabilidades $\hat{\mathbf{p}}$:

$$L = -\sum_{k=1}^{K} y_k \log \hat{p}_k$$

con:

$$\hat{p}_k = \frac{e^{z_k}}{\sum_j e^{z_j}}$$

Minimizar esta pérdida aumenta la probabilidad asignada a la clase correcta y penaliza con fuerza las predicciones confiadas que resultan equivocadas. La clase predicha puede ser $\arg\max_k \hat{p}_k$, pero las probabilidades requieren calibración si se usarán para decisiones de riesgo.

Descenso de gradiente sobre un mini lote

El entrenamiento ajusta los pesos para reducir la pérdida. En cada actualización, estima el gradiente con un mini lote $\mathcal{B}_t \subset \mathcal{D}_{\text{train}}$:

$$L_{\mathcal{B}_t}(\theta) = \frac{1}{|\mathcal{B}_t|} \sum_{(x_i, y_i) \in \mathcal{B}_t} \ell\big(f_\theta(x_i), y_i\big)$$

$$\theta_{t+1} = \theta_t - \eta \nabla_\theta L_{\mathcal{B}_t}(\theta_t)$$

El gradiente indica el cambio local de la pérdida. La tasa η controla el tamaño de cada actualización.

Mini lotes, actualizaciones y épocas

- Un **mini lote** $\mathcal{B}_t$ produce una estimación estocástica del gradiente.
- Una **actualización** modifica θ una vez, después de procesar un mini lote.
- Una **época** recorre una vez todos los ejemplos de entrenamiento.

Optimizadores: SGD y Adam

Ambos usan el gradiente del mini lote, pero actualizan los pesos de forma distinta.

SGD

- Usa el gradiente actual y una tasa de aprendizaje común, η.
- Es simple y requiere poca memoria adicional.
- Es sensible a η: una tasa inadecuada puede ralentizar o desestabilizar el ajuste.

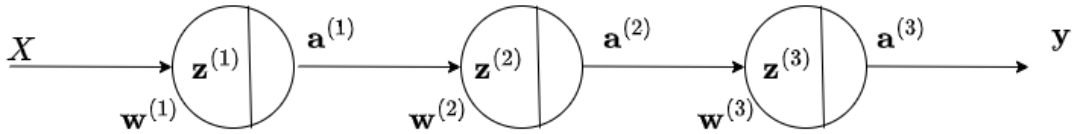
Adam

- Combina promedios de gradientes recientes y de sus cuadrados.
- Adapta el tamaño del paso para cada parámetro.
- Suele ser un buen punto de partida, aunque requiere más memoria y no siempre supera a SGD.

Hay otros optimizadores, como RMSprop y Adagrad. Por ahora veremos **SGD y Adam**.

Red de ejemplo para calcular gradientes

La entrada x pasa por tres neuronas conectadas en serie. Cada una calcula un puntaje $z^{(l)}$ y una activación $a^{(l)}$. La tercera activación es la predicción $\hat{y} = a^{(3)}$.



Propagación hacia adelante

En cada neurona, primero calculamos el puntaje ponderado y luego aplicamos la activación:

$$
\begin{aligned}
z^{(1)} &= x w^{(1)}, & a^{(1)} &= h^{(1)}(z^{(1)}) \\
z^{(2)} &= a^{(1)} w^{(2)}, & a^{(2)} &= h^{(2)}(z^{(2)}) \\
z^{(3)} &= a^{(2)} w^{(3)}, & \hat{y} = a^{(3)} &= h^{(3)}(z^{(3)})
\end{aligned}
$$

La salida de una neurona se convierte en la entrada de la siguiente.

Pérdida de una predicción

Para una observación con valor esperado y, usamos el error cuadrático:

$$J = \left(a^{(3)} - y\right)^2$$

La salida se obtiene con $a^{(3)} = h^{(3)}\left(z^{(3)}\right)$. Calcularemos cuánto cambia J al modificar cada peso.

Derivada respecto al peso de salida

Para actualizar $w^{(3)}$ usamos la regla de la cadena:

$$\frac{\partial J}{\partial w^{(3)}} = \frac{\partial z^{(3)}}{\partial w^{(3)}} \cdot \frac{\partial a^{(3)}}{\partial z^{(3)}} \cdot \frac{\partial J}{\partial a^{(3)}}$$

Calculamos cada término:

$$\frac{\partial J}{\partial a^{(3)}} = \frac{\partial}{\partial a^{(3)}}(a^{(3)} - y)^2 = 2(a^{(3)} - y)$$

$$\frac{\partial a^{(3)}}{\partial z^{(3)}} = \frac{\partial}{\partial z^{(3)}} h^{(3)}(z^{(3)}) = \left(h^{(3)}\right)'(z^{(3)})$$

$$\frac{\partial z^{(3)}}{\partial w^{(3)}} = \frac{\partial}{\partial w^{(3)}}(a^{(2)} \cdot w^{(3)}) = a^{(2)}$$

Gradiente del peso de salida

Entonces:

$$\frac{\partial J}{\partial w^{(3)}} = a^{(2)} \cdot \left(h^{(3)}\right)'(z^{(3)}) \cdot 2(a^{(3)} - y)$$

Gradiente de una capa anterior

Para actualizar $w^{(2)}$ usamos la regla de la cadena:

$$\frac{\partial J}{\partial w^{(2)}} = \frac{\partial z^{(2)}}{\partial w^{(2)}} \cdot \frac{\partial a^{(2)}}{\partial z^{(2)}} \cdot \frac{\partial z^{(3)}}{\partial a^{(2)}} \cdot \boxed{\frac{\partial a^{(3)}}{\partial z^{(3)}} \cdot \frac{\partial J}{\partial a^{(3)}}}$$

Las últimas dos derivadas ya se calcularon para $\frac{\partial J}{\partial w^{(3)}}$:

- $\frac{\partial a^{(3)}}{\partial z^{(3)}} = \left(h^{(3)}\right)'(z^{(3)})$
- $\frac{\partial J}{\partial a^{(3)}} = 2(a^{(3)} - y)$

Estas derivadas se **propagan hacia atrás** para calcular gradientes de capas anteriores.

Propagación de la señal de error

Definimos la señal de error:

$$\delta^{(l)} = \frac{\partial J}{\partial z^{(l)}}$$

Para la capa de salida:

$$\delta^{(3)} = 2(a^{(3)} - y) \cdot \left(h^{(3)}\right)'(z^{(3)})$$

Para las capas anteriores:

$$\delta^{(2)} = \delta^{(3)} w^{(3)} \left(h^{(2)}\right)'(z^{(2)}), \qquad \delta^{(1)} = \delta^{(2)} w^{(2)} \left(h^{(1)}\right)'(z^{(1)})$$

Gradiente de cada peso

Finalmente:

$$\frac{\partial J}{\partial w^{(l)}} = \delta^{(l)} \cdot a^{(l-1)}$$

donde $a^{(0)} = x$ es la entrada.

Resumen del entrenamiento de la red

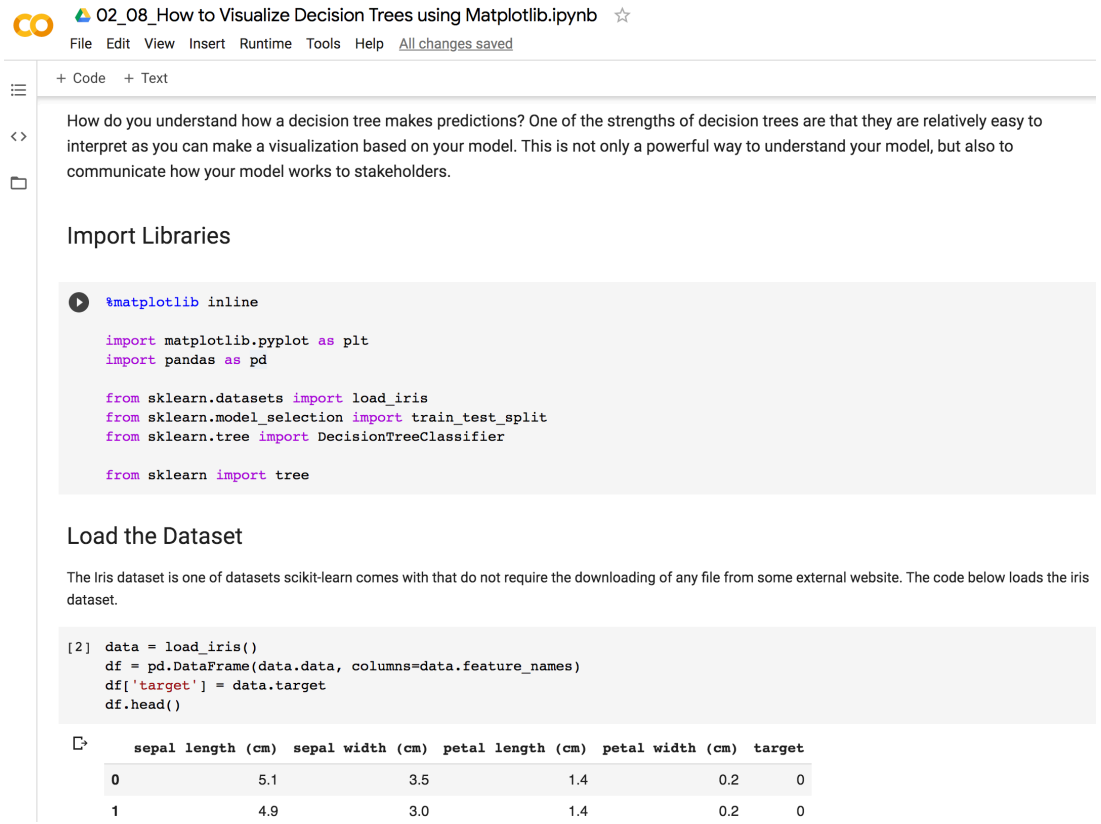
Paso	Operación
1. Propagación	Calcular $z^{(l)}, a^{(l)}$ desde la entrada hasta la salida
2. Pérdida	$J = (a^{(3)} - y)^2$
3. Señal de salida	$\delta^{(3)} = 2(a^{(3)} - y)\left(h^{(3)}\right)'(z^{(3)})$
4. Retropropagación	$\delta^{(l)} = \delta^{(l+1)} w^{(l+1)} \left(h^{(l)}\right)'(z^{(l)})$
5. Gradientes y actualización	$\frac{\partial J}{\partial w^{(l)}} = \delta^{(l)} a^{(l-1)}$, $w^{(l)} \leftarrow w^{(l)} - \eta \frac{\partial J}{\partial w^{(l)}}$

La señal de salida se reutiliza para obtener los gradientes de las capas anteriores.

Entorno de ejecución: Google Colaboratory

Colab permite escribir y ejecutar notebooks Python desde el navegador.

- No requiere configuración local inicial.
- Facilita compartir el trabajo.
- Puede ofrecer acceso a GPU.



Ciclo de entrenamiento y evaluación en PyTorch

1. **Definir** un `nn.Module`: capas, tamaños y activaciones.
2. **Preparar** un `Dataset` y sus lotes con `DataLoader`.
3. **Entrenar** con `model.train()`, `loss.backward()` y `optimizer.step()`.
4. **Validar/evaluar** con `model.eval()` y `torch.no_grad()`.

En clasificación con varias clases, la salida tiene una neurona por clase. `CrossEntropyLoss` recibe los *logits* directamente: no agregamos `softmax` dentro del modelo.

> El ciclo es explícito, pero PyTorch tampoco decide por nosotros la arquitectura, la partición ni la métrica relevante.

Datos para el ejemplo de clasificación

Ejemplo de clasificación binaria con datos artificiales: cuatro características, dos clases y una partición de 80 % para entrenamiento y 20 % para validación.

```
import numpy as np
X = np.random.default_rng(7).normal(size=(1000, 4)).astype("float32")
y = (X[:, 0] + X[:, 1] > 0).astype("int64")
x_tr, x_val = X[:800], X[800:]
y_tr, y_val = y[:800], y[800:]
```

Ejemplo Pytorch

Datos y modelo

```
train = DataLoader(TensorDataset(
    torch.from_numpy(x_tr), torch.from_numpy(y_tr)),
    batch_size=32, shuffle=True)
model = nn.Sequential(
    nn.Linear(4, 16), nn.ReLU(), nn.Linear(16, 2))
loss_fn = nn.CrossEntropyLoss()
opt = torch.optim.Adam(model.parameters())
```

- `TensorDataset` empareja ejemplos y etiquetas. `DataLoader` agrupa 32 ejemplos y los mezcla.
- `Linear(4, 16)` → ReLU → `Linear(16, 2)` produce dos *logits*. `CrossEntropyLoss` y Adam definen la pérdida y el ajuste.

Entrenamiento y evaluación

```
for _ in range(3):
    model.train()
    for xb, yb in train:
        opt.zero_grad()
        loss = loss_fn(model(xb), yb)
        loss.backward()
        opt.step()
model.eval()
with torch.no_grad():
    pred = model(torch.from_numpy(x_val)).argmax(1)
acc = (pred == torch.from_numpy(y_val)).float().mean()
```

- Por lote, `zero_grad()` limpia los gradientes, `backward()` los calcula y `step()` actualiza los pesos.
- `eval()` fija el modo de evaluación. `no_grad()` omite gradientes. `argmax()` predice la clase.

Comparación: PyTorch y TensorFlow/Keras

PyTorch: ciclo explícito

```
import torch
from torch import nn
from torch.utils.data import DataLoader, TensorDataset

train = DataLoader(TensorDataset(
    torch.from_numpy(x_tr), torch.from_numpy(y_tr)),
    batch_size=32, shuffle=True)
valid = DataLoader(TensorDataset(
    torch.from_numpy(x_val), torch.from_numpy(y_val)), batch_size=32)
model = nn.Sequential(
    nn.Linear(4, 16), nn.ReLU(), nn.Linear(16, 2))
loss_fn = nn.CrossEntropyLoss()
opt = torch.optim.Adam(model.parameters())
for _ in range(3):
    model.train()
    for xb, yb in train:
        opt.zero_grad()
        loss_fn(model(xb), yb).backward()
        opt.step()
model.eval()
with torch.no_grad():
    acc = sum((model(x).argmax(1) == y).sum().item()
              for x, y in valid) / len(y_val)
print("validación:", acc)
```

TensorFlow/Keras: ciclo encapsulado

```
import tensorflow as tf

train = tf.data.Dataset.from_tensor_slices(
    (x_tr, y_tr)).shuffle(800).batch(32)
valid = tf.data.Dataset.from_tensor_slices(
    (x_val, y_val)).batch(32)
model = tf.keras.Sequential([
    tf.keras.layers.Input((4,)),
    tf.keras.layers.Dense(16, activation="relu"),
    tf.keras.layers.Dense(2)])
model.compile(optimizer="adam",
    loss=tf.keras.losses.SparseCategoricalCrossentropy(
        from_logits=True), metrics=["accuracy"])
model.fit(train, validation_data=valid, epochs=3, verbose=0)
print("validación:", model.evaluate(valid, verbose=0)[1])
```

Ambos usan la misma red y pérdida. PyTorch muestra los pasos de actualización. Keras los reúne en `fit`.

Diagnóstico de subajuste y sobreajuste

Subajuste

- Error alto en entrenamiento.
- Modelo o características insuficientes.
- Optimización incompleta.

Sobreajuste

- Error bajo en entrenamiento.
- Brecha alta en validación.
- El modelo memoriza particularidades.

Generalizar es mantener un error bajo en datos no usados para ajustar los pesos. Las curvas de aprendizaje comparan el error de entrenamiento y validación durante el ajuste.

Penalización de la función objetivo

Penalización L_2:

$$L_{total} = L_{datos} + \lambda \sum_j w_j^2$$

Penalización L_1:

$$L_{total} = L_{datos} + \lambda \sum_j |w_j|$$

El coeficiente λ controla el compromiso entre ajustar los datos y restringir la complejidad efectiva del modelo.

Estrategias de regularización y control de capacidad

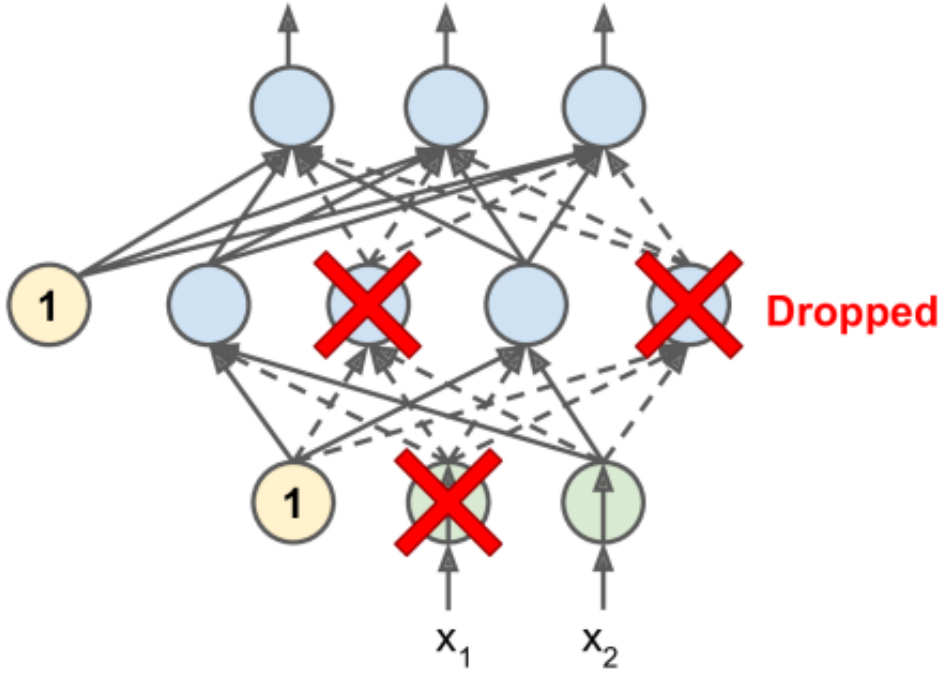
Además de las penalizaciones, el protocolo puede incluir:

- **Dropout:** perturbar activaciones durante el entrenamiento.
- **Detención temprana:** elegir la época según validación.
- **Aumento de datos:** incorporar transformaciones que preservan la etiqueta.
- **Reducción de capacidad:** limitar capas, unidades o parámetros.

L_1 puede anular pesos, mientras que L_2 reduce su magnitud. Ajustamos λ y las demás decisiones con validación.

Regularización en PyTorch.

Dropout



¿Cómo funciona Dropout?

Durante el entrenamiento, Dropout desactiva al azar algunas activaciones. Así, la red no depende siempre de las mismas neuronas y aprende representaciones más robustas. En evaluación se usan todas las neuronas.

Early Stop

La detención temprana monitorea la pérdida de validación y conserva el instante de mejor desempeño:

$$t^* = \arg\min_t L_{\text{val}}(t)$$

- Evita seguir ajustando cuando comienza el sobreajuste.
- Puede detener el entrenamiento antes del presupuesto máximo.
- Requiere decidir el criterio usando validación.

La prueba se evalúa una vez, después de fijar t^* y todas las decisiones.

Arquitecturas según la estructura de los datos

La arquitectura incorpora una **hipótesis inductiva**: una restricción que aprovecha la estructura esperada de la entrada.

Estructura de la entrada	Hipótesis incorporada	Arquitectura inicial
Vector sin orden espacial o temporal	Interacciones globales entre características	Red densa o MLP
Secuencia ordenada	El pasado relevante afecta el presente	RNN, LSTM o GRU
Imagen o grilla	Patrones locales pueden aparecer en distintas posiciones	CNN

La arquitectura, su capacidad y su costo computacional son hiperparámetros. Se comparan con el mismo protocolo de validación, no solo con la pérdida de entrenamiento.

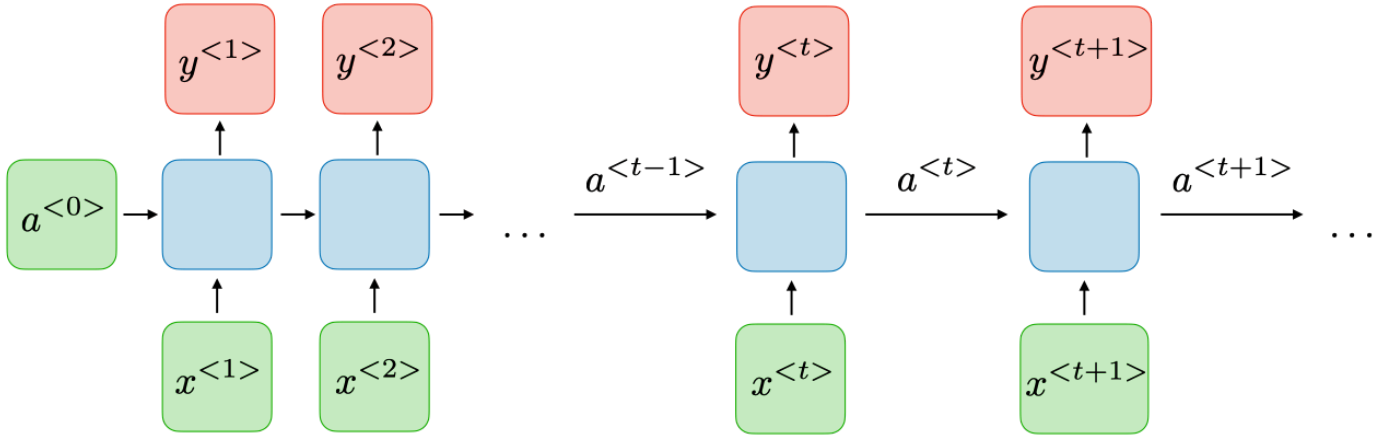
LSTM: memoria controlada por compuertas

Una celda LSTM recibe la entrada de un instante y actualiza su memoria. Al reutilizarla en cada instante y pasar su estado al siguiente, se forma una red recurrente.

- Sus compuertas deciden qué información conservar y cuál olvidar.
- Otras compuertas regulan qué información nueva guardar y qué parte entregar como salida.

Así, la unidad puede mantener patrones relevantes durante varios pasos.

Red Recurrente



Una RNN procesa una secuencia paso a paso. En cada instante combina la entrada actual con información de los pasos anteriores.

La misma unidad se reutiliza a lo largo de la secuencia. Puede ser una unidad simple, una celda LSTM o una GRU.

Pronóstico un paso adelante con una LSTM

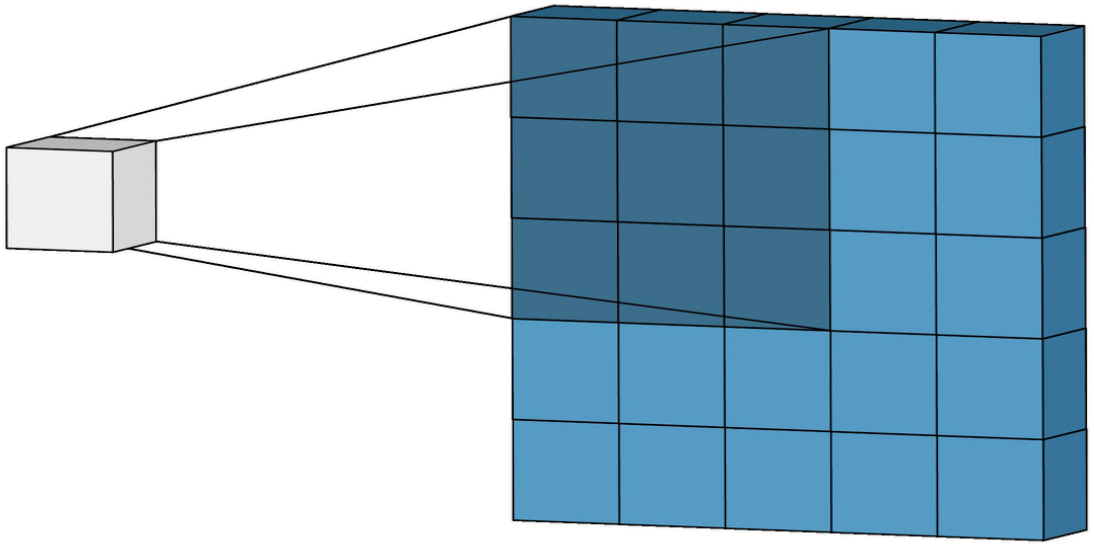
El notebook de LSTM en PyTorch pronostica manchas solares un mes hacia adelante:

$$(x_{t-w}, \ldots, x_{t-1}) \xrightarrow{\text{LSTM}} \mathbf{h}_t \xrightarrow{\text{Dense}} \hat{x}_t$$

- Separa entrenamiento y prueba por fecha, sin mezclar observaciones futuras.
- Ajusta el escalamiento exclusivamente sobre el período de entrenamiento.
- Construye ejemplos mediante ventanas temporales de ancho w.
- Compara la señal observada y la predicción en el período futuro.

La ventana w define cuánto pasado recibe el modelo. El horizonte de predicción y el método de validación temporal deben corresponder a la decisión real que se quiere apoyar.

Convolución: filtros locales



Un filtro recorre vecindarios de la imagen. La animación muestra cómo cada posición de entrada contribuye a una región de salida.

Neurona convolucional: suma local y activación

En una CNN, cada neurona analiza una pequeña región de la imagen. Combina sus píxeles con un filtro y produce una activación en el mapa de características.

- El filtro recorre distintas posiciones de la imagen y reutiliza los mismos pesos.
- Cada filtro puede responder a un patrón, como un borde o una textura.
- Varios filtros producen mapas distintos que las capas siguientes combinan.

Región de píxeles → filtro → activación

¿Qué calcula un filtro convolucional?

- Un **filtro** o *kernel* es una pequeña matriz de coeficientes que recorre la imagen
- En cada posición multiplica los coeficientes por los píxeles vecinos y suma los productos

$$y = \sum_{q \in \text{parche}} k_q x_q + b$$

Símbolo	Significado
y	Activación en la posición actual
x_q	Píxel de entrada en la posición q
k_q	Peso del filtro en la misma posición
b	Sesgo que se suma al resultado

El filtro se reutiliza en otras posiciones y produce un **mapa de características**

¿Cómo aprende la red los filtros?

- Los valores del filtro K y su sesgo b son **parámetros entrenables** de la capa convolucional
- La red los inicializa al comienzo y no hace falta indicar manualmente "filtro de bordes" o "filtro de textura"
- Durante el entrenamiento la pérdida mide el error y la retropropagación calcula cómo cambiar cada coeficiente
- El optimizador aplica la actualización:

$$K \leftarrow K - \eta \frac{\partial \mathcal{L}}{\partial K}$$

Respuesta: los valores del filtro son los pesos de la convolución

- No son parámetros adicionales a los pesos de la red
- La red aprende también los pesos de sus otras capas

Pooling: reducción de resolución espacial



Agrupación por máximo: conserva la activación máxima de una región.

Agrupación por promedio: conserva el promedio de la región.

El *pooling* reduce la resolución espacial y el costo, pero también descarta información sobre la ubicación exacta de los patrones.

El tamaño de ventana y el paso determinan cuánto se reduce la representación. Una reducción agresiva puede eliminar detalles relevantes. No siempre conviene usarla en tareas que requieren localización precisa.

Arquitectura CNN para clasificar CIFAR-10

El notebook CNN de CIFAR-10 en PyTorch usa imágenes de 32×32 píxeles, tres canales y diez clases:

$$\mathrm{Conv2D} \to \mathrm{MaxPooling} \to \mathrm{Conv2D} \to \mathrm{MaxPooling} \to \mathrm{Conv2D} \to \mathrm{Dense} \to \mathrm{softmax}$$

Las convoluciones extraen patrones locales. Las capas densas combinan la representación final para clasificar. La profundidad, los canales y la regularización deben seleccionarse con validación.

> El ejemplo entrena durante un número fijo de épocas y evalúa en prueba. Para comparar arquitecturas, conviene añadir validación y mantener el mismo protocolo.

Diseño experimental antes de entrenar una red

1. **Definir** la tarea, población objetivo, decisión y métrica de éxito.
2. **Construir** una línea base simple que establezca un desempeño de referencia.
3. **Separar** datos antes de ajustar transformaciones, respetando tiempo, grupos y clases cuando corresponda.
4. **Fijar** presupuesto de cómputo, semillas, arquitectura inicial y espacio de hiperparámetros.

> Una red más profunda solo es mejor si supera una línea base bajo una comparación justa y con un costo aceptable.

Selección, prueba y reporte del experimento

1. **Entrenar** cada configuración solo con entrenamiento y monitorear validación.
2. **Seleccionar** hiperparámetros y época con una regla definida antes de mirar prueba.
3. **Reentrenar**, si el protocolo lo contempla, usando únicamente los datos autorizados.
4. **Evaluar** el conjunto de prueba una vez y reportar la métrica acordada.
5. **Analizar** errores por clase, segmento, condición y costo operacional.

Al comparar modelos, informe también variabilidad entre semillas, tiempo de entrenamiento, recursos usados y limitaciones de los datos. Un único valor de exactitud no caracteriza completamente un experimento.

Notebook de referencia: Fashion MNIST con PyTorch

El notebook principal `notebook/04_clasificacion_fashion_mnist_pytorch.ipynb`:

- usa Fashion MNIST: imágenes en escala de grises de prendas, agrupadas en diez categorías.
- normaliza imágenes y separa entrenamiento, validación y prueba.
- entrena una red densa con `DataLoader`, *dropout* y un ciclo explícito.
- usa detención temprana sobre la pérdida de validación.
- calcula matriz de confusión, exactitud global y exactitud por clase.

El ejemplo sigue la API de PyTorch y deja explícitas las semillas, la partición y el dispositivo utilizado. El notebook equivalente en TensorFlow queda como material de comparación en el anexo.

Principios para diseñar y evaluar redes neuronales

- El punto de partida es la tarea, los datos y la métrica. La arquitectura es una hipótesis que debe validarse.
- El entrenamiento minimiza una pérdida promedio: retropropagación calcula gradientes y el optimizador actualiza parámetros.
- La separación entre entrenamiento, validación y prueba protege la estimación de generalización.
- Regularización, capacidad y detención temprana se eligen como parte del protocolo experimental.
- Las RNN modelan secuencias. Las CNN aprovechan patrones locales.

Anexo: TensorFlow y Keras

TensorFlow es una biblioteca para trabajar con tensores, diferenciación automática y modelos de aprendizaje profundo.

Su API de alto nivel más conocida es **Keras**, que permite describir modelos secuenciales y delegar el ciclo de entrenamiento en métodos como `fit`.

En esta clase, PyTorch es la implementación principal. TensorFlow queda como referencia.

Anexo: equivalencias entre PyTorch y TensorFlow/Keras

Concepto	PyTorch	TensorFlow / Keras
Tensor	`torch.Tensor`	`tf.Tensor`
Capa densa	`nn.Linear`	`keras.layers.Dense`
Modelo	`nn.Module`	`keras.Model` / `Sequential`
Pérdida para varias clases	`nn.CrossEntropyLoss`	`SparseCategoricalCrossentropy(from_logits=True)`
Optimizador	`torch.optim.Adam`	`keras.optimizers.Adam`
Lotes de datos	`DataLoader`	`tf.data.Dataset`
Entrenar/evaluar	`model.train()` / `model.eval()`	`training=True` / `training=False`

Anexo: notebooks equivalentes para el mismo experimento

Los dos notebooks implementan el mismo experimento:

- `04_clasificacion_fashion_mnist_pytorch.ipynb`: versión principal de la clase.
- `04_clasificacion_fashion_mnist_tensorflow.ipynb`: versión de referencia para este anexo.

Ambos normalizan Fashion MNIST y usan las mismas particiones. La red tiene 128 unidades ReLU, *dropout* de 0.25 y 10 *logits*. Los dos aplican detención temprana y reportan métricas por clase.

Las métricas pueden variar por la inicialización y el hardware, incluso si se conserva el mismo protocolo.

Referencias y material de profundización

- Material original de IPD434 (referencia histórica, con ejemplos TensorFlow): `04_RedesNeuronales.ipynb`.
- Notebook principal reproducible: `notebook/04_clasificacion_fashion_mnist_pytorch.ipynb`.
- Notebook de referencia para el anexo: `notebook/04_clasificacion_fashion_mnist_tensorflow.ipynb`.
- A. Géron, *Hands On Machine Learning with Scikit Learn, Keras & TensorFlow*, O'Reilly.
- Stanford, *CS229 Machine Learning Notes*.
- I. Goodfellow, Y. Bengio y A. Courville, *Deep Learning*, MIT Press, 2016.
- C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
- F. Chollet, *Deep Learning with Python*, 2.ª ed., Manning, 2021.
- PyTorch, documentación oficial de módulos, optimización y carga de datos.
- TensorFlow/Keras, documentación oficial de clasificación y modelos secuenciales.